

Laporan Mata Kuliah Praktikum Kualitas Perangkat Lunak

UJIAN AKHIR SEMESTER

disusun untuk memenuhi tugas mata kuliah
Praktikum Kualitas Perangkat Lunak

oleh:

MUHAMMAD AIDIL FITRAH

2308107010035



**JURUSAN INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS SYIAH KUALA**

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengujian perangkat lunak merupakan salah satu tahapan kritis dalam siklus pengembangan perangkat lunak (Software Development Life Cycle/SDLC). Tanpa pengujian yang sistematis, kualitas dan keandalan sebuah sistem tidak dapat dijamin. Mata kuliah Kualitas Perangkat Lunak (KPL) mengajarkan mahasiswa untuk mampu merancang, membangun, dan menguji aplikasi backend secara terstruktur.

Pada Ujian Akhir Semester (UAS) ini, praktikan diminta untuk membangun sebuah backend REST API menggunakan framework pilihan dan melakukan pengujian terhadap setiap endpoint yang dibuat menggunakan aplikasi Postman. Postman adalah salah satu tool pengujian API yang paling banyak digunakan oleh para developer karena kemudahannya dalam mengirimkan berbagai jenis HTTP request dan memvalidasi respons dari server.

Project yang dibangun dalam laporan ini adalah Book Management API, yaitu sistem manajemen data buku berbasis REST API yang dibangun menggunakan Node.js dan framework Express.js. API ini menyediakan enam endpoint yang mencakup operasi Create, Read, Update, dan Delete (CRUD) lengkap, melebihi batas minimum empat endpoint yang ditentukan.

1.2 Tujuan Pengujian

Pengujian ini dilakukan dengan beberapa tujuan utama, yaitu:

- Memverifikasi bahwa setiap endpoint API berfungsi sesuai dengan spesifikasi yang telah dirancang.
- Memastikan bahwa status code HTTP yang dikembalikan oleh server sudah sesuai dan benar (200 OK, 201 Created, 400 Bad Request, 404 Not Found).
- Memvalidasi bahwa struktur data respons JSON sudah konsisten dan terstruktur dengan baik.

- Menguji mekanisme validasi input pada endpoint yang menerima data dari client (POST, PUT, PATCH).
- Menguji kemampuan penanganan error (Error Handling) sistem ketika menerima request yang tidak valid.

1.3 Ruang Lingkup Pengujian

Pengujian dilakukan secara lokal (localhost) menggunakan Postman. Ruang lingkup pengujian mencakup enam endpoint utama yang terdiri dari dua metode tambahan di luar ketentuan minimum, yaitu GET by ID dan PATCH.

BAB II

DESKRIPSI SISTEM

2.1 Gambaran Umum Sistem

Book Management API adalah sebuah layanan web backend yang menyediakan antarmuka pemrograman aplikasi (API) berbasis arsitektur REST (Representational State Transfer) untuk mengelola data buku. Sistem ini memungkinkan client (seperti aplikasi frontend atau tools seperti Postman) untuk melakukan operasi CRUD terhadap data buku yang tersimpan di dalam memori server (in-memory storage).

Sistem ini dibangun dengan pendekatan yang sederhana namun solid, menggunakan array JavaScript sebagai mock database untuk menghindari kebutuhan instalasi database eksternal. Setiap endpoint dilengkapi dengan mekanisme validasi input dan penanganan error yang terstruktur.

2.2 Teknologi yang Digunakan

Komponen	Teknologi / Versi	Keterangan
Runtime	Node.js (LTS)	Lingkungan eksekusi JavaScript di sisi server
Framework	Express.js ^4.19.2	Framework web minimalis untuk routing dan middleware
Middleware	CORS ^2.8.5	Mengizinkan Cross-Origin Resource Sharing
Dev Tools	Nodemon ^3.1.0	Auto-restart server saat ada perubahan file kode
Testing Tool	Postman	Pengujian dan validasi endpoint REST API
Storage	In-Memory (Array)	Penyimpanan data sementara di RAM server

2.3 Struktur Proyek

Berikut adalah struktur direktori dari proyek yang telah dibuat:

```
UAS KPL/  
├── .gitignore  
├── package.json  
└── server.js
```

```
└─ routes/
   └─ books.js
```

Penjelasan masing-masing file:

- `.gitignore` — Mendefinisikan file dan folder yang diabaikan oleh Git (node_modules).
- `package.json` — File konfigurasi project Node.js yang mendefinisikan metadata proyek, script, dan daftar dependensi.
- `server.js` — Entry point utama aplikasi. Mengonfigurasi server Express, mendaftarkan middleware, routing, serta global error handler.
- `routes/books.js` — Berisi definisi in-memory database (array buku) beserta seluruh logika routing dan handler untuk enam endpoint CRUD.

2.4 Deskripsi Endpoint API

Base URL yang digunakan selama pengujian adalah: `http://localhost:3000`

No	Metode	Endpoint	Fungsi	Status Sukses
1	POST	<code>/api/books</code>	Menambah data buku baru	201 Created
2	GET	<code>/api/books</code>	Mengambil semua data buku	200 OK
3	GET	<code>/api/books/:id</code>	Mengambil satu data buku berdasarkan ID	200 OK
4	PUT	<code>/api/books/:id</code>	Memperbarui seluruh data buku (full update)	200 OK
5	PATCH	<code>/api/books/:id</code>	Memperbarui sebagian data buku (partial update)	200 OK
6	DELETE	<code>/api/books/:id</code>	Menghapus data buku berdasarkan ID	200 OK

2.5 Struktur Data Buku

Setiap objek buku dalam sistem memiliki struktur data JSON sebagai berikut:

```
{
  "id"      : Number   (auto-generated, unik)
  "judul"   : String   (wajib, tidak boleh kosong)
```

```
"penulis": String    (wajib, tidak boleh kosong)
"tahun"   : Number   (wajib, tipe number)
"genre"   : String    (wajib, tidak boleh kosong)
"stok"    : Number    (wajib, tipe number)
}
```

BAB III

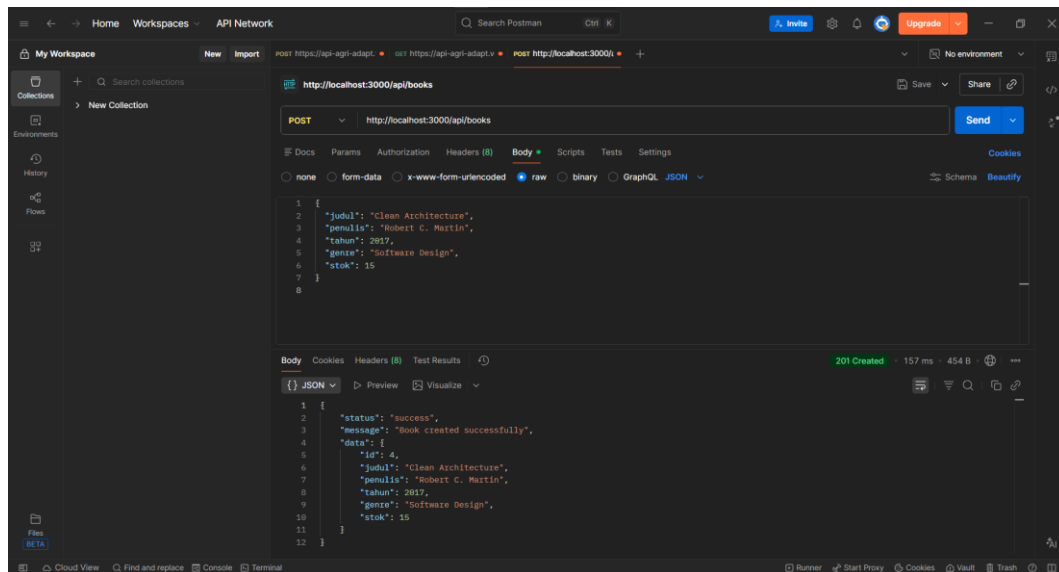
HASIL PENGUJIAN

3.1 Lingkungan Pengujian

Pengujian dilakukan dengan konfigurasi lingkungan sebagai berikut:

Parameter	Keterangan
Sistem Operasi	Windows 10
Tools Pengujian	Postman
URL Pengujian	http://localhost:3000
Server	Node.js dengan Express.js (npm run dev)
Format Request Body	application/json (JSON)

3.2 Pengujian 1 – POST /api/books (Create)

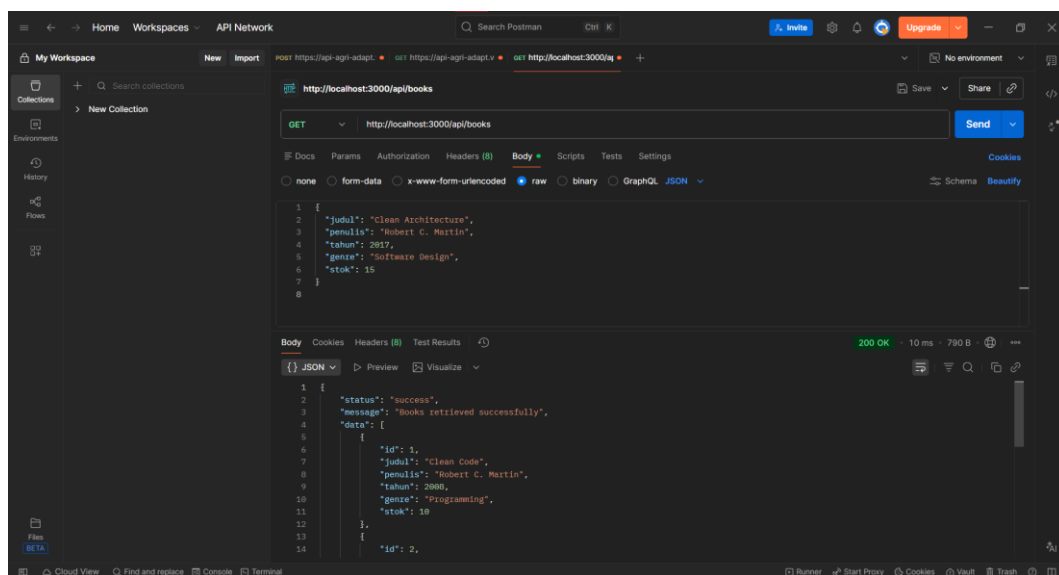


Gambar 3.1. Hasil pengujian POST /api/books (Create)

Analisis Hasil: Pengujian endpoint POST /api/books berhasil dilakukan. Sistem menerima data buku baru yang dikirimkan melalui request body dalam format JSON, kemudian menyimpannya ke dalam in-memory storage. Server mengembalikan respons dengan status code 201 Created beserta objek data buku yang baru saja disimpan, termasuk ID yang di-generate secara otomatis oleh sistem.

3.3 Pengujian 2 – GET /api/books (Read All)

- **Tujuan:** Menguji kemampuan sistem untuk mengembalikan seluruh data buku yang tersimpan dalam database.
- **Metode HTTP:** GET
- **URL:** `http://localhost:3000/api/books`
- **Headers:** Tidak diperlukan
- **Request Body:** Tidak ada (GET request tidak membutuhkan body)
- **Expected Status Code:** 200 OK
- **Expected Response:** Array JSON yang berisi seluruh objek buku yang terdaftar di dalam sistem (data awal + buku baru yang baru di-POST).

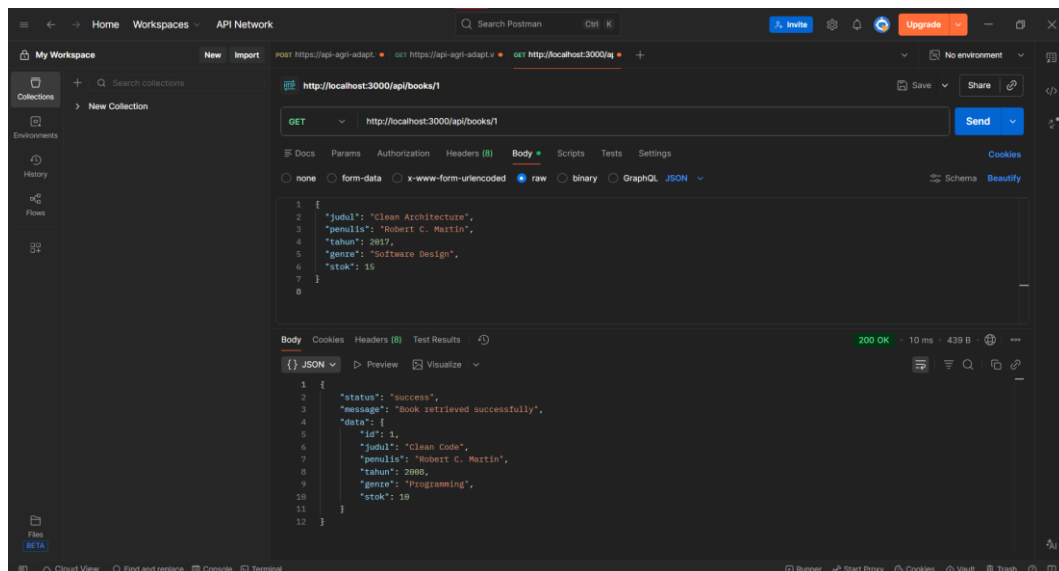


Gambar 3.2. Hasil pengujian GET /api/books (Read All)

Analisis Hasil: Pengujian endpoint GET /api/books berhasil dilakukan. Sistem mengembalikan seluruh data buku yang tersimpan dalam bentuk array JSON. Respons server mencakup data buku awal (seed data) yang sudah ada sejak server pertama kali dijalankan, beserta data buku baru yang sebelumnya telah ditambahkan melalui pengujian POST. Status code 200 OK dikembalikan sesuai ekspektasi.

3.4 Pengujian 3 – GET /api/books/:id (Read by ID)

- **Tujuan:** Menguji kemampuan sistem untuk mengembalikan satu data buku spesifik berdasarkan parameter ID yang diberikan pada URL.
- **Metode HTTP:** GET
- **URL:** http://localhost:3000/api/books/1
- **Headers:** Tidak diperlukan
- **Request Body:** Tidak ada
- **Expected Status Code:** 200 OK
- **Expected Response:** Objek JSON dari satu buku yang memiliki ID = 1.



Gambar 3.3. Hasil pengujian GET /api/books/:id (Read by ID)

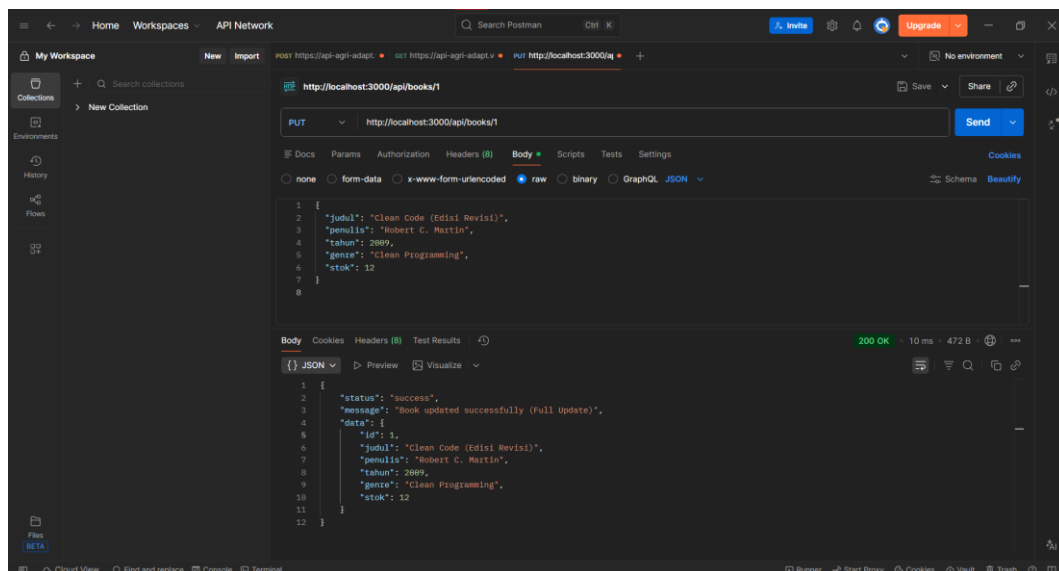
Analisis Hasil: Pengujian endpoint GET /api/books/:id berhasil dilakukan. Sistem berhasil menelusuri data dalam array berdasarkan parameter ID yang diberikan pada URL path, lalu mengembalikan objek buku yang sesuai. Endpoint ini merupakan endpoint tambahan (di luar ketentuan minimum 4 endpoint) yang memberikan kemampuan untuk mengakses detail satu entitas secara spesifik, sehingga API ini lebih RESTful dan fungsional.

3.5 Pengujian 4 – PUT /api/books/:id (Full Update)

- **Tujuan:** Menguji kemampuan sistem untuk mengganti seluruh data buku yang ada berdasarkan ID dengan data baru yang dikirimkan.
- **Metode HTTP:** PUT
- **URL:** http://localhost:3000/api/books/1
- **Headers:** Content-Type: application/json
- Request Body yang dikirimkan (seluruh field wajib diisi):

```
{
  "judul" : "Clean Code (Edisi Revisi)",
  "penulis": "Robert C. Martin",
  "tahun" : 2009,
  "genre" : "Clean Programming",
  "stok" : 12
}
```

- **Expected Status Code:** 200 OK
- **Expected Response:** Objek JSON dari buku ID 1 dengan data yang sudah diperbarui sesuai request body.



Gambar 3.4. Hasil pengujian PUT /api/books/:id (Full Update)

Analisis Hasil: Pengujian endpoint PUT /api/books/:id berhasil dilakukan. Metode PUT bersifat full update, artinya seluruh field data buku akan digantikan dengan data baru yang dikirimkan pada request body. Sistem memvalidasi bahwa

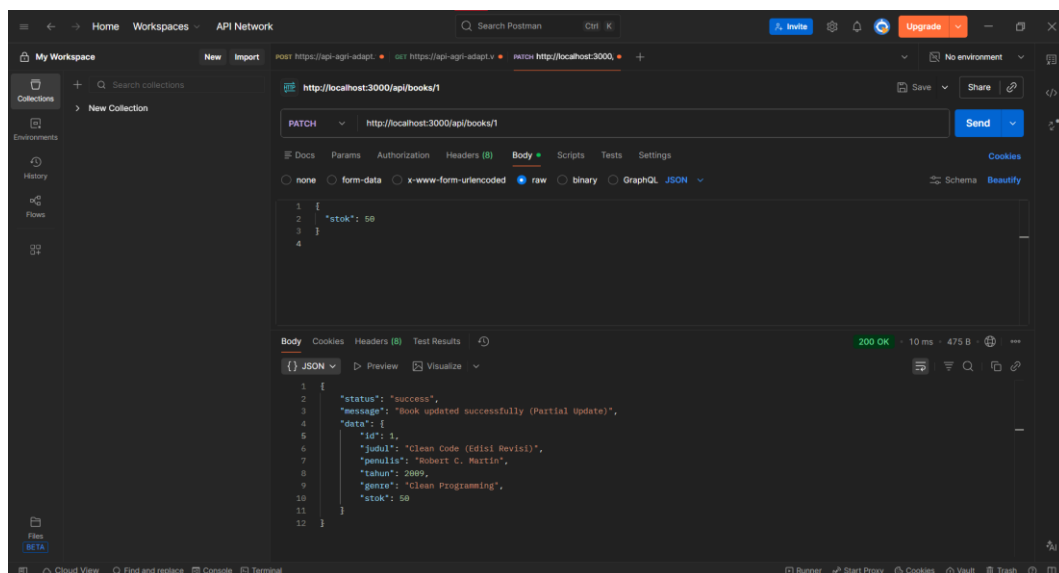
seluruh field wajib tersedia sebelum melakukan pembaruan. Server mengembalikan objek buku yang telah diperbarui dengan status code 200 OK.

3.6 Pengujian 5 – PATCH /api/books/:id (Partial Update)

- **Tujuan:** Menguji kemampuan sistem untuk memperbarui sebagian field data buku saja (partial update) tanpa harus mengirimkan semua field.
- **Metode HTTP:** PATCH
- **URL:** http://localhost:3000/api/books/1
- **Headers:** Content-Type: application/json
- Request Body yang dikirimkan (hanya field yang ingin diubah):

```
{  
  "stok": 50  
}
```

- **Expected Status Code:** 200 OK
- **Expected Response:** Objek JSON dari buku ID 1 dengan field 'stok' yang sudah diperbarui menjadi 50, sedangkan field lainnya tetap tidak berubah.



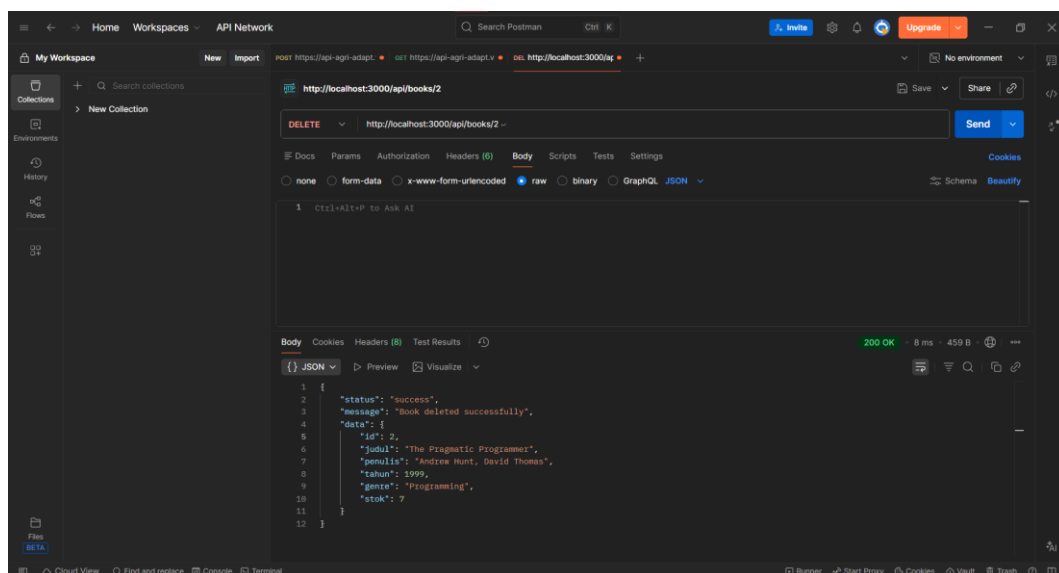
Gambar 3.5. Hasil pengujian PATCH /api/books/:id (Partial Update)

Analisis Hasil: Pengujian endpoint PATCH /api/books/:id berhasil dilakukan. Berbeda dengan PUT yang menggantikan seluruh data, PATCH hanya

memperbarui field yang dikirimkan pada request body. Field-field lain yang tidak disebutkan akan tetap mempertahankan nilai aslinya. Ini menjadikan PATCH lebih efisien untuk skenario pembaruan parsial seperti hanya mengubah stok buku tanpa mengubah data bibliografinya.

3.7 Pengujian 6 – DELETE /api/books/:id (Delete)

- **Tujuan:** Menguji kemampuan sistem untuk menghapus data buku dari database berdasarkan ID yang diberikan.
- **Metode HTTP:** DELETE
- **URL:** `http://localhost:3000/api/books/2`
- **Headers:** Tidak diperlukan
- **Request Body:** Tidak ada
- **Expected Status Code:** 200 OK
- **Expected Response:** Objek JSON dari data buku yang berhasil dihapus sebagai konfirmasi penghapusan.



Gambar 3.6. Hasil pengujian DELETE /api/books/:id (Delete)

Analisis Hasil: Pengujian endpoint DELETE /api/books/:id berhasil dilakukan. Sistem menelusuri data pada array berdasarkan ID yang diberikan, kemudian menghapusnya menggunakan operasi splice(). Sistem mengembalikan

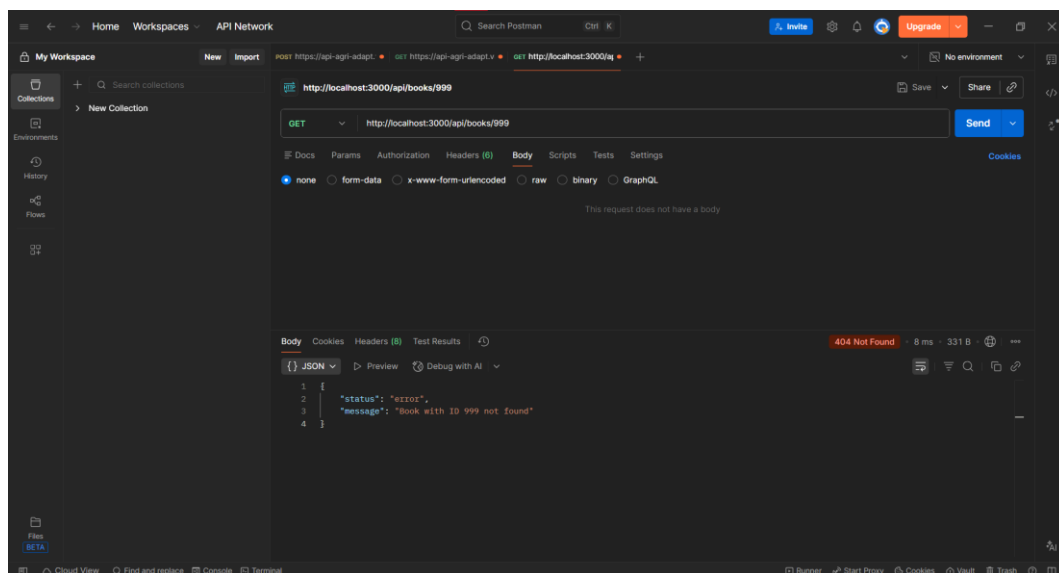
objek dari data yang berhasil dihapus sebagai bukti dan konfirmasi kepada client bahwa operasi penghapusan telah berhasil dilakukan.

3.8 Pengujian 7 – Error Handling (Nilai Tambah)

Pengujian error handling dilakukan untuk memastikan sistem memberikan respons yang informatif dan sesuai saat menerima request yang tidak valid atau ketika resource yang diminta tidak ditemukan.

3.8.1 Skenario A – GET Resource Tidak Ditemukan (404 Not Found)

- **URL:** `http://localhost:3000/api/books/999`
- **Metode HTTP:** GET
- **Expected Status Code:** 404 Not Found
- **Expected Response:** Objek JSON dengan status 'error' dan pesan yang menyatakan bahwa buku dengan ID tersebut tidak ditemukan.



Gambar 3.7. Hasil pengujian GET Resource Tidak Ditemukan (404 Not Found)

Analisis Hasil: Sistem berhasil menangani permintaan terhadap ID yang tidak terdaftar dengan mengembalikan status code 404 Not Found beserta pesan

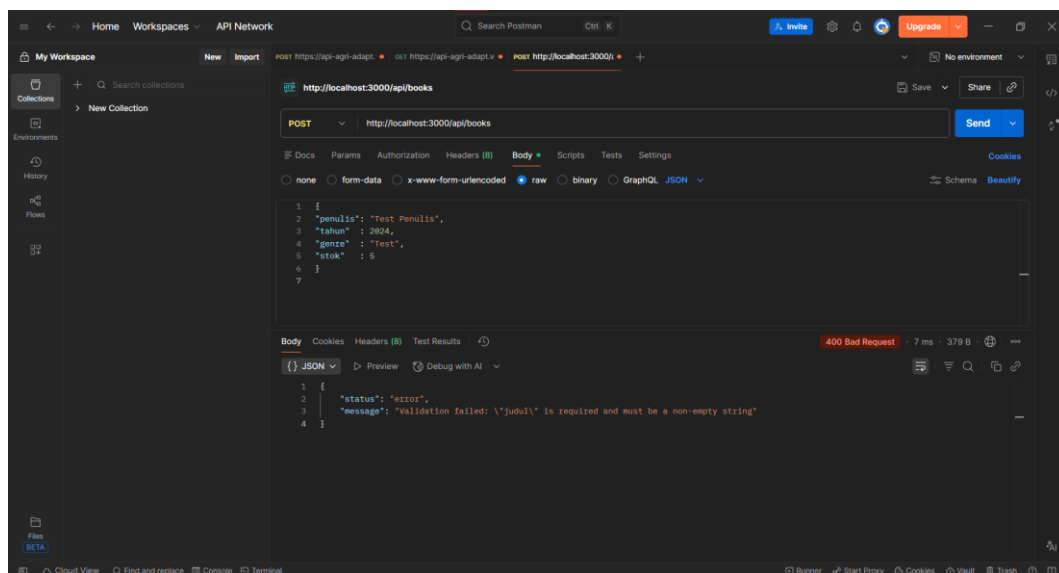
error yang informatif. Ini membuktikan bahwa sistem memiliki mekanisme validasi ketersediaan resource yang baik.

3.8.2 Skenario B – POST dengan Field Kosong (400 Bad Request)

- **URL:** `http://localhost:3000/api/books`
- **Metode HTTP:** POST
- Request Body yang dikirimkan (field 'judul' dikosongkan sengaja):

```
{
  "penulis": "Test Penulis",
  "tahun"   : 2024,
  "genre"   : "Test",
  "stok"    : 5
}
```

- **Expected Status Code:** 400 Bad Request
- **Expected Response:** Objek JSON dengan status 'error' dan pesan validasi yang menyatakan bahwa field 'judul' wajib diisi.



Gambar 3.8. Hasil pengujian POST dengan Field Kosong (400 Bad Request)

Analisis Hasil: Sistem berhasil menangani request dengan data yang tidak lengkap melalui mekanisme validasi input yang telah diimplementasikan. Server

mengembalikan status code 400 Bad Request beserta pesan error yang spesifik menyebutkan field mana yang tidak valid, sehingga memudahkan developer atau client untuk memperbaiki request-nya.

3.9 Rekapitulasi Hasil Pengujian

No	Endpoint	Metode	Expected Code	Actual Code	Status
1	/api/books	POST	201 Created	201 Created	PASS
2	/api/books	GET	200 OK	200 OK	PASS
3	/api/books/:id	GET	200 OK	200 OK	PASS
4	/api/books/:id	PUT	200 OK	200 OK	PASS
5	/api/books/:id	PATCH	200 OK	200 OK	PASS
6	/api/books/:id	DELETE	200 OK	200 OK	PASS
7A	/api/books/999	GET	404 Not Found	404 Not Found	PASS
7B	/api/books	POST (invalid)	400 Bad Request	400 Bad Request	PASS

BAB IV

KESIMPULAN

4.1 Kesimpulan

Berdasarkan seluruh rangkaian pengujian yang telah dilakukan terhadap Book Management API menggunakan Postman, dapat ditarik kesimpulan sebagai berikut:

- Seluruh endpoint yang dibangun (POST, GET, GET by ID, PUT, PATCH, dan DELETE) berfungsi dengan benar dan mengembalikan respons JSON yang terstruktur sesuai dengan spesifikasi yang telah dirancang.
- Mekanisme validasi input yang diterapkan pada endpoint POST, PUT, dan PATCH berjalan dengan baik, terbukti dari keberhasilan pengujian error handling yang mengembalikan status code 400 Bad Request ketika data yang dikirimkan tidak valid.
- Mekanisme penanganan resource tidak ditemukan (404 Not Found) berjalan dengan baik saat client mengakses ID yang tidak terdaftar dalam sistem.
- API yang dibangun melebihi batas minimum empat endpoint (CRUD) dengan menambahkan endpoint GET by ID dan metode PATCH, sehingga API bersifat lebih lengkap dan mengikuti prinsip RESTful API yang baik.
- Total delapan skenario pengujian yang dilakukan semuanya menghasilkan status PASS, menunjukkan bahwa sistem berjalan secara stabil dan sesuai harapan.

LAMPIRAN

Link Repository: [Github](#)